

Secure State Estimation of a Cyber-Physical System under Sparse Sensor Attacks: A Comparative Analysis

Gabriele Tedeschi
s358478
s358478@studenti.polito.it

Lorenzo Maggio
s360725
s360725@studenti.polito.it

Francesco Favellato
s364140
s364140@studenti.polito.it

Abstract—This paper investigates the problem of secure state estimation in dynamic Cyber-Physical Systems (CPSs) subject to sparse and constant sensor attacks. We consider a CPS modeled as a discrete-time LTI system and evaluate the performance of four algorithms designed to estimate the system state and the attack vector. In particular, we implement and compare Online Proximal Gradient Descent (O-PGD), Online Proximal Alternating Minimization (O-PAM), and their aggregated variants, AO-PGD and AO-PAM. Numerical experiments conducted on a networked system comprising 50 states and 20 sensors, 4 of which are compromised, demonstrate the effectiveness of the proposed techniques.

I. INTRODUCTION

In this paper we consider a discrete-time linear time-invariant (LTI) system subject to sensor corruptions, modeled as follows:

$$x(k+1) = Ax(k) \quad (1)$$

$$y(k) = Cx(k) + a \quad (2)$$

where $x(k) \in \mathbb{R}^n$ denotes the system state vector at time step k , and $y(k) \in \mathbb{R}^q$ represents the measurement vector obtained from the sensor network. The matrices $A \in \mathbb{R}^{n \times n}$ and $C \in \mathbb{R}^{q \times n}$ define the system dynamics and the measurement topology, respectively. Crucially, we assume that the full state is not directly measurable ($C \neq I$), necessitating a state reconstruction approach. The vector $a \in \mathbb{R}^q$ represents an exogenous, constant, and sparse sensor attack. The assumption of sparsity is highly representative of practical cyber-security scenarios: due to resource constraints or localized security barriers, an attacker can typically compromise only a limited subset of the available sensors, leaving the others uncorrupted.

In this context, the primary objective of this work is to achieve secure state estimation. This involves reconstructing the true evolution of the state of the system $x(k)$ despite the presence of adversarial corruption, while simultaneously identifying the exact subset of compromised sensors (the support of the attack vector a).

This paper explores and assesses four online algorithms based on proximal optimization, specifically designed for sparse attacks detection and state estimation. The performance of these methodologies is rigorously analyzed and compared through extensive numerical simulations. The evaluation is conducted across a suite of performance metrics, including

computational execution time, state estimation error, support attack error, as well as false positive and false negative detection rates.

II. ANALYSIS OF SYSTEM DYNAMICS

A. Structural Properties and Graph Topology

To understand the behavior of the Cyber-Physical System, we first perform a structural analysis of the system matrix A . Numerical evaluations show that $A \in \mathbb{R}^{50 \times 50}$ is a row-stochastic matrix, meaning that the elements of each row sum up to exactly one. This mathematical property allows us to interpret A as a Perron matrix related to a consensus protocol among the agents. By isolating the off-diagonal entries, we extract the network's adjacency matrix A_{adj} , which defines the directional communication links between nodes. Mathematically, a consensus matrix P is related to the graph's Adjacency Matrix through the relation:

$$P = I - \epsilon(D - A_{\text{adj}}) \quad (3)$$

where $\epsilon > 0$ is a small scaling parameter.

By analyzing the directed graph generated from this topology (Figure 1), we observe two key characteristics. First, the network contains exactly two *Leader* nodes, which are characterized by an in-degree of zero. This implies that these nodes act as independent information sources that influence the network without receiving information from other agents. Second, the graph possesses at least one directed spanning tree rooted at these leaders, which guarantees that information propagates from the leaders to the rest of the system.

B. Eigenvalue Analysis and Limitations of the Luenberger Observer

Next, we investigate the properties of matrix A by computing its eigenvalues. The analysis reveals the presence of a dominant eigenvalue $\lambda_{1,2} = 1$ with an algebraic multiplicity of two. The multiplicity of this eigenvalue directly matches the number of independent leader nodes identified in the graph topology.

From a dynamical systems perspective, this implies the existence of two constant natural modes that do not decay over time. Hence, the CPS is not observable by using a standard Luenberger observer.

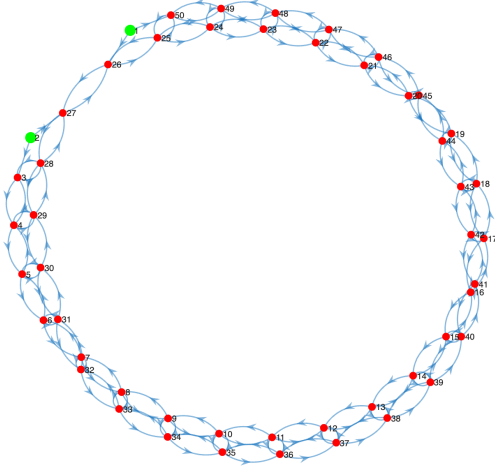


Fig. 1. Directed Graph of the CPS's topology with 2 Leaders

III. SECURE STATE ESTIMATION ALGORITHMS

We now analyze the four distinct approaches employed to address the problem of secure state estimation in the presence of sparse sensor attacks. All the algorithms are based on the minimization of a cost function $F(x, a)$, implemented through an online framework that iteratively updates the estimates of the initial state $x(0)$ and the attack vector a . The evaluation of the performance of each algorithm has been implemented through several numerical simulations in Matlab with simulation time $T = 500$ time steps. In our experiments, the hyper-parameters were empirically tuned through a trial-and-error procedure to achieve the optimal trade-off across the evaluated performance metrics.

The metrics used for each algorithm are the followings:

- **State estimation error:** Measures the Mean Squared Error (MSE) normalized by the state dimension n :

$$\frac{\|A^k \hat{x}(k) - x(k)\|_2^2}{n} \quad (4)$$

- **Support attack error:** Evaluates the mismatch between the estimated and the true attack support:

$$\sum_{j=1}^q |1(\hat{a}_j(k) \neq 0) - 1(\tilde{a}_j \neq 0)| \quad (5)$$

where the indicator function $1(v) = 1$ if the condition v is true, and 0 otherwise.

- **False positives:** Computes the number of uncorrupted sensors incorrectly classified as attacked:

$$\sum_{j=1}^q [1(\hat{a}_j(k) \neq 0) - 1(\tilde{a}_j \neq 0)]_+ \quad (6)$$

- **False negatives:** Computes the number of attacked sensors missed by the estimator:

$$\sum_{j=1}^q [1(\hat{a}_j(k) \neq 0) - 1(\tilde{a}_j \neq 0)]_- \quad (7)$$

A. Online Proximal Gradient Descent (O-PGD)

The Online Proximal Gradient Descent (O-PGD) algorithm try to solve the secure state estimation problem by iteratively minimizing a cost function at each time step. The estimation of the initial state $x(0)$ relies on a standard gradient descent update step applied to the smooth and differentiable term. On the other hand, the estimation of the attack vector a requires a proximal approach. The ℓ_1 -norm regularization penalty, which is essential to correctly enforce the sparsity of the attack, is a non-differentiable function. For this reason, the algorithm computes this update step analytically by using the soft-thresholding operator.

The mathematical formulation of the algorithm is shown in the following:

$$\hat{x}(k+1) = \hat{x}(k) - \nu(CA^k)^\top (CA^k \hat{x}(k) + \hat{a}(k) - y(k)) \quad (8)$$

$$\hat{a}(k+1) = \mathbb{S}_{\nu\lambda} [\hat{a}(k) - \nu(CA^k \hat{x}(k) + \hat{a}(k) - y(k))] \quad (9)$$

where $\mathbb{S}_{|\cdot|}$ is the soft-thresholding (or proximal) operator, ν and λ are hyper-parameters that tunes respectively the value of the update step of the gradient descent, and the threshold used by the proximal operator.

Figure 2 shows the results of the numerical simulation using the following hyper-parameters: $\lambda = 0.3$, $\nu = 0.125$.

As observed in the graphs, the algorithm successfully reconstructs all state variables, achieving a state estimation error of approximately 10^{-3} after 130 iterations. Simultaneously, it correctly identifies the compromised sensors, driving the support attack error to zero within 88 iterations. Furthermore, the trend of the false positives closely mirrors that of the support attack error: during the initial iterations, the algorithm tends to overestimate the presence of attacks, which are subsequently filtered out by the soft-thresholding operator. Consequently, the false negatives remain essentially zero throughout the entire process.

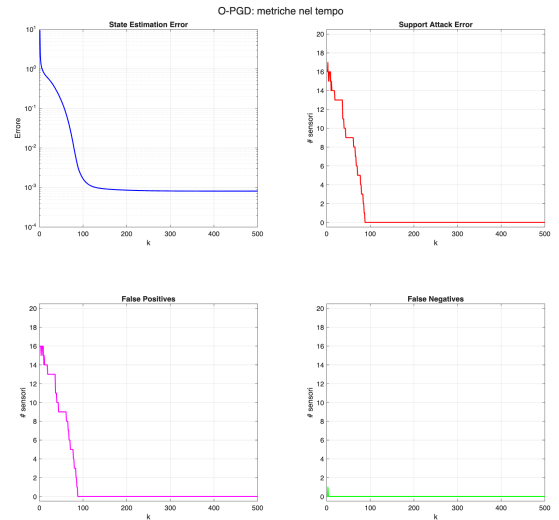


Fig. 2. O-PGD Evaluation Metrics

B. Online Proximal Alternating Minimization (O-PAM)

The Online Proximal Alternating Minimization (O-PAM) algorithm adopts a fundamentally different approach. Rather than relying on gradient descent steps to minimize the cost function, it solves the problem through an alternating minimization scheme. Specifically, by keeping the attack vector a fixed, it first minimizes the objective with respect to x to update the state estimate. Subsequently, it leverages this newly updated estimate to minimize the function with respect to a . The core distinction of this method lies in the matrix inversion step and the inclusion of a proximal regularization term, which acts as a “cost to move” penalty. The hyper-parameter μ , when appropriately tuned, ensures the stability and convergence of the algorithm.

The mathematical formulation of the O-PAM updates is expressed as follows:

$$\hat{x}(k+1) = ((CA^k)^\top CA^k + \mu I)^{-1} (\mu \hat{x}(k) + (CA^k)^\top (y(k) - \hat{a}(k))) \quad (10)$$

$$\hat{a}(k+1) = \frac{1}{1+\mu} \mathbb{S}_\lambda (y(k) - CA^k \hat{x}(k+1) + \mu \hat{a}(k)) \quad (11)$$

The performance evaluation of the O-PAM algorithm is presented in Figure 3, utilizing the hyperparameters:

$$\lambda = 0.3, \mu = 0.1$$

An analysis of the performance plots reveals that the State Estimation Error exhibits a rapid initial convergence, before stabilizing at a value of 10^{-3} after 250 iterations. Similarly, the Support Attack Error goes to zero by the 12nd iteration and maintains this optimal steady-state throughout the remainder of the simulation. The dynamics concerning false positives and false negatives mirror the behavior discussed in the previous section.

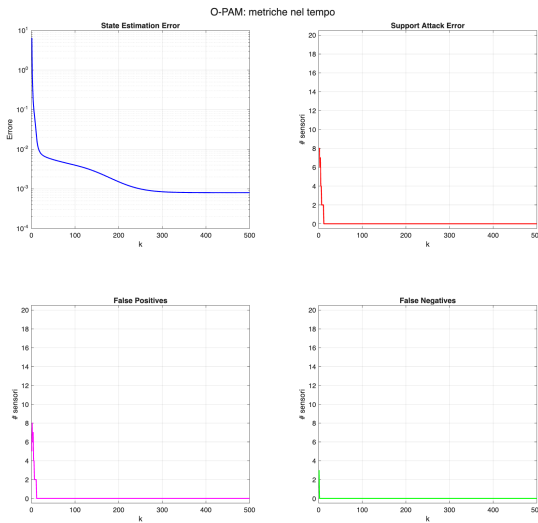


Fig. 3. O-PAM Evaluation Metrics

C. Aggregated Online Proximal Gradient Descent (AO-PGD)

The Aggregated Online Proximal Gradient Descent (AO-PGD) algorithm introduces a variation to the standard online

approach by aggregating historical data. Instead of relying only on the current measurement $y(k)$ to perform the update, AO-PGD aggregates the past measurements. Specifically, the algorithm updates the initial state and attack estimates by computing the average of the gradients evaluated from time step 0 up to the current step k .

The mathematical formulation of the AO-PGD updates is defined as follows:

$$\hat{x}(k+1) = \hat{x}(k) - \frac{\nu}{k+1} \sum_{j=0}^k (CA^j)^\top (CA^j \hat{x}(k) + \hat{a}(k) - y(j)) \quad (12)$$

$$\hat{a}(k+1) = \mathbb{S}_{\nu\lambda} \left[\hat{a}(k) - \nu \left(\frac{1}{k+1} \sum_{j=0}^k CA^j \hat{x}(k) + \hat{a}(k) - y(j) \right) \right] \quad (13)$$

Figure 4 shows the results of the numerical simulation using the following hyper-parameters: $\lambda = 0.3, \nu = 0.125$

An examination of the graphs reveals that the aggregation strategy actually degrades the convergence speed in this specific scenario. The State Estimation Error barely reaches the value of 10^{-3} throughout the 500 steps. Similarly, the algorithm requires approximately 109 iterations to completely eliminate the Support Attack Error. Because the algorithm averages past measurements, the historical accumulation delays the correction of the initial overestimations. Consequently, the false positive rate persists for a longer transient phase before converging. False negatives remain strictly at zero throughout the entire execution.

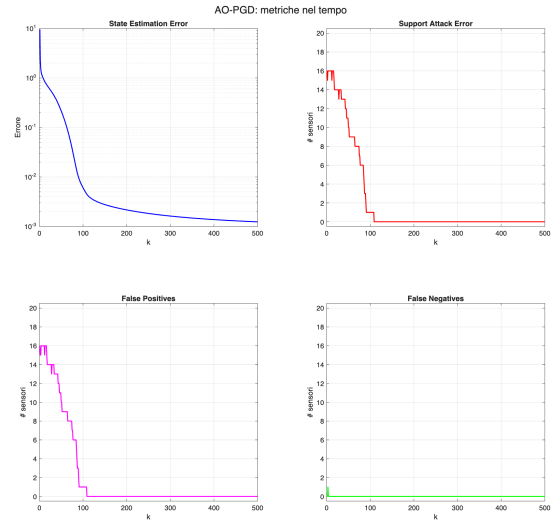


Fig. 4. AO-PGD Evaluation Metrics

D. Aggregated Online Proximal Alternating Minimization (AO-PAM)

The Aggregated Online Proximal Alternating Minimization (AO-PAM) algorithm represents an extension of the standard alternating scheme, designed to incorporate the entire history of past observations up to the current time step k . Instead of solving the proximal minimization based only on the instantaneous measurement $y(k)$, AO-PAM sequentially updates the

initial state and the attack vector by minimizing an objective function that aggregates all historical measurement matrices and vectors.

Since a formal derivation of the aggregated alternating scheme is not available in the literature, we analytically extended the O-PAM algorithm by reproducing the aggregation methodology originally applied to the O-PGD algorithm. For the state estimation, the algorithm finds the vector x that minimizes the aggregated residual errors while remaining close to the previous estimate $\hat{x}(k)$:

$$\begin{aligned}\hat{x}(k+1) &= \arg \min_{x \in \mathbb{R}^n} \left\{ \frac{1}{k+1} \sum_{j=0}^k \frac{1}{2} \|C^j x + \hat{a}(k) - y(j)\|_2^2 + \frac{\mu}{2} \|x - \hat{x}(k)\|_2^2 \right\} \\ &= \left(\frac{1}{k+1} \sum_{j=0}^k (C^j)^T C^j + \mu I \right)^{-1} \left(\mu \hat{x}(k) + \frac{1}{k+1} \sum_{j=0}^k (C^j)^T (y(j) - \hat{a}(k)) \right)\end{aligned}\quad (14)$$

Subsequently, using the newly computed $\hat{x}(k+1)$, the attack vector $\hat{a}(k+1)$ is updated by solving an ℓ_1 -regularized minimization problem:

$$\begin{aligned}\hat{a}(k+1) &= \arg \min_{a \in \mathbb{R}^m} \left\{ \frac{1}{k+1} \sum_{j=0}^k \frac{1}{2} \|C^j \hat{x}(k+1) + a - y(j)\|_2^2 + \lambda \|a\|_1 + \frac{\mu}{2} \|a - \hat{a}(k)\|_2^2 \right\} \\ &= \mathbb{S}_{\frac{\lambda}{1+\mu}} \left[\frac{1}{1+\mu} \left(\mu \hat{a}(k) + \frac{1}{k+1} \sum_{j=0}^k (y(j) - C^j \hat{x}(k+1)) \right) \right]\end{aligned}\quad (15)$$

The performance of the AO-PAM algorithm is evaluated using the following hyperparameters select by our tuning procedure: $\lambda = 0.3$, $\mu = 0.1$.

Figure 5 presents the individual evaluation metrics for the AO-PAM framework. The numerical results indicate that while the algorithm successfully achieves state reconstruction and drive the metrics to their optimal values, the convergence speed is visibly penalized by the aggregation process. The State Estimation Error experiences a very slow transient decay, reaching the 10^{-3} threshold only near the end of the simulation ($k = 500$). The Support Attack Error and false positives strictly converge to zero at approximately iteration 11, while false negatives remain identically zero.

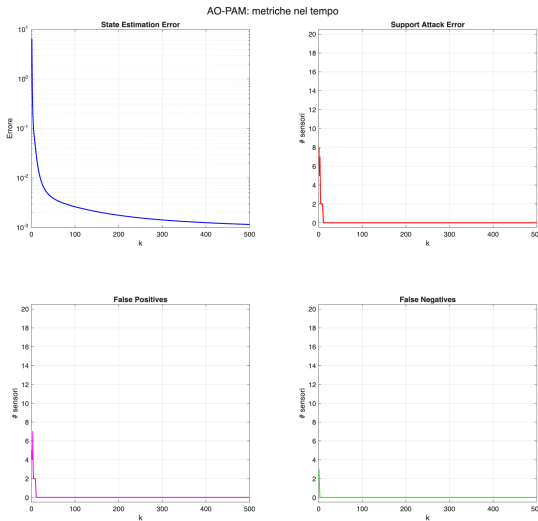


Fig. 5. AO-PAM Evaluation Metrics

IV. EXPERIMENTAL RESULTS AND COMPARATIVE ANALYSIS

This section evaluates the performance of the four implemented algorithms. First, we discuss the empirical hyperparameter tuning. Next, we compare the algorithms in terms of state estimation and attack reconstruction, focusing on the convergence speed of the State Estimation Error and Support Attack Error metrics. Finally, we analyze their computational complexity and overall execution time.

A. Hyperparameter Tuning

The hyperparameters were empirically tuned through a trial-and-error procedure to achieve an optimal performance setup across the evaluated metrics. The final values were selected to guarantee the best trade-off between the State Estimation Error and the Support Attack Error. For consistency in the comparison, the parameters were kept uniform across paired algorithmic frameworks (i.e., O-PGD matched with AO-PGD, and O-PAM matched with AO-PAM). Although custom fine-tuning for each individual algorithm could further reduce the state estimation error, it would simultaneously degrade the accuracy of the attack support identification:

- λ : By increasing the value of the regularization parameter, the algorithms tend to systematically underestimate the presence of attacks. Consequently, this leads to a significant rise in false negatives and a proportional decrease in false positives. Furthermore, the State Estimation Error converges to a higher steady-state value. On the opposite, decreasing λ produces the opposite behavior: the estimators become overly sensitive and tend to overestimate the attacks, misclassifying uncorrupted measurements as threats.
- μ : For the algorithms belonging to the PAM family, this parameter acts as a numerical "friction" that ensures convergence. By increasing its value, we observe that the State Estimation Error shows a longer transient phase before reaching steady-state, while the Support Attack Error takes significantly longer to settle at zero. On the contrary, decreasing this parameter introduces instability, leading to visible oscillations in both the state estimation tracking and the attack support identification.
- ν : For the PGD family of algorithms, this parameter defines the step size of the gradient descent update. Increasing its value leads to larger update steps, which introduces the risk of overshooting the optimal minimum and causing the algorithm to diverge. On the other side, setting this parameter too low ensures that the algorithm will eventually identify the minimum, but at the cost of a drastically slower convergence rate.

B. State Estimation and Attack Reconstruction

Here, we present a qualitative comparison of the State Estimation Error and Support Attack Error across the four algorithms, acknowledging that they operate based on distinct fundamental hyper-parameters (i.e., ν and μ).

Figure 6 illustrates the joint behavior of the State Estimation Error. It is immediately apparent that the PAM family rapidly suppresses the error during the initial iterations, exhibiting a steep descent. In contrast, the PGD family shows a more gradual decay, although it ultimately settles at a slightly lower steady-state error value, keeping in mind the logarithmic scale of the y-axis.

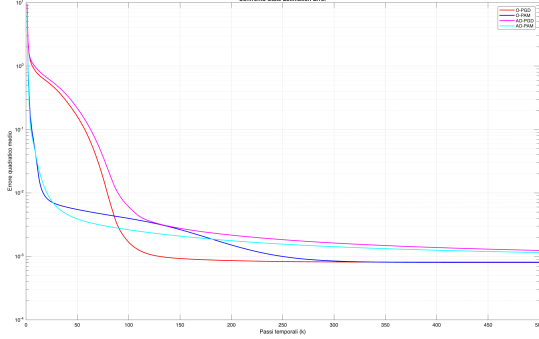


Fig. 6. State Estimation Error comparison

Figure 7 displays the Support Attack Error trajectories. In this context, the PAM family is clearly superior in terms of convergence speed, almost instantly identifying which sensors are subjected to external attacks.

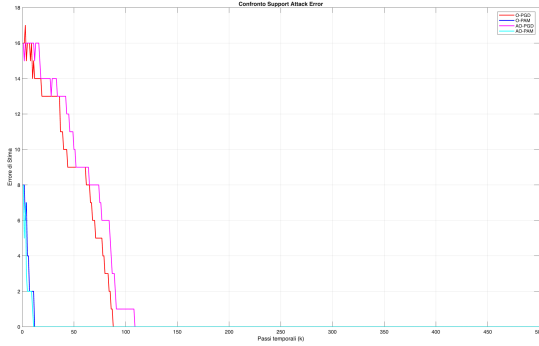


Fig. 7. Support Attack Error comparison

Figure 8 shows the final estimated magnitudes of the attack vector. As observed, there is a slight discrepancy between the estimated and true values. This deviation is a consequence of the soft-thresholding operator, which shrinks the non-zero components by a maximum of $\pm\lambda$, introducing a small bias into the final estimate.

C. Computational Complexity and Execution Time

Finally, regarding the overall execution times, while the performance gap among the four algorithms would become significantly more pronounced over a larger number of iterations, the current results already indicate a clear trend. The algorithms belonging to the PGD family are computationally faster than their PAM counterparts. This difference is due to mathematical operations required at each update step: while the PGD frameworks rely on gradient steps involving standard matrix-vector multiplications, the PAM formulation requires

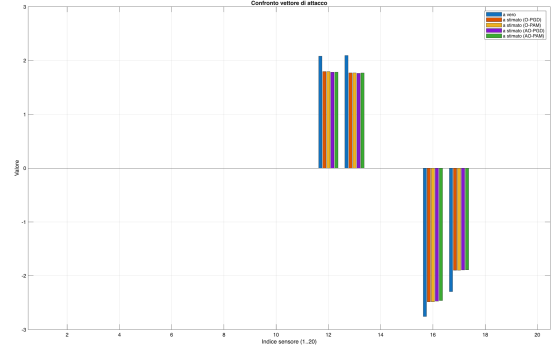


Fig. 8. Attack: real value vs estimated value

a full matrix inversion at every iteration to update the state estimate. This inversion step introduces a substantially higher computational cost, slowing down the alternating minimization approach.

We intentionally omit the absolute execution times from our analysis, as these values are highly hardware-dependent and subject to run-to-run fluctuations based on the processor's current workload. Instead, our comparison focuses on the relative computational overhead, which confirms that the PGD family requires significantly less execution time than the PAM frameworks.

V. CONCLUSION

This paper addressed secure state estimation in Cyber-Physical Systems under sparse sensor attacks, a scenario where traditional estimators fail due to unobservable constant modes. We implemented and compared four proximal optimization algorithms: O-PGD, AO-PGD, O-PAM, and AO-PAM. Our analysis highlighted that optimal performance heavily relies on hyper-parameter tuning to efficiently balance the State Estimation Error and the Support Attack Error. When properly calibrated, the alternating minimization (PAM) algorithms detects compromised sensors in significantly fewer iterations than the gradient-based (PGD) methods.

Furthermore, the experimental results revealed that historical data aggregation slowed down the algorithmic performance within the context of our specific case study. Contrary to general expectations, the mathematical averaging in AO-PGD and AO-PAM significantly delayed the error decay, rendering them less effective than their standard, non-aggregated counterparts. It is crucial to emphasize that this empirical observation is strictly constrained to our investigated scenario and the uniform hyperparameter configuration adopted to ensure a fair comparison.